Patrones de diseño aplicados

Sistema de Préstamos Interbibliotecarios

TP Integrador 2026 · Tema 1: Biblioteca · Grupo 93
Desarrollo de Aplicaciones con Objetos · UTN – Facultad Regional Córdoba

Noviembre de 2026

1. Introducción

La consigna exige aplicar **al menos dos patrones de diseño**. El sistema aplica **State** (en dos lugares: el ciclo de vida del remito y el de la reserva) e **Iterator** (el recorrido del catálogo). Son los patrones que el grupo definió en el diagrama de clases, y cada uno resuelve un problema concreto del dominio de la biblioteca. Este documento explica, para cada uno, qué problema resuelve en el TP, cómo se implementó y por qué se eligió.

Cada patrón se presenta con el esquema de la cátedra: **problema**, **solución** (qué clase cumple cada rol del patrón), **implementación** (extractos reales del código con archivo y número de línea), **justificación**, **consecuencias** y **verificación** (los tests automáticos que lo comprueban).

Resumen

Patrón	Tipo	Clases	Problema que resuelve
State	Comportamiento	Remito, EstadoRemito, Pendiente, Empaquetado, Enviado, Entregado	Qué operación logística admite un remito según la etapa del envío (ida y retorno).
State	Comportamiento	Reserva, EstadoReserva y sus cuatro estados	Ciclo de una reserva: confirmar, cumplir o cancelar según corresponda.
Iterator	Comportamiento	Catalogo, Iterador, IteradorEjemplares, IteradorEjemplaresDisponibles	Recorrer el catálogo completo o solo lo disponible de una sede, sin exponer la colección.

Dónde están los patrones

El sistema está organizado en capas: el **dominio** (`biblioteca/modelos/`), los **casos de uso** (`biblioteca/servicios.py`), la **persistencia** en SQLite (`biblioteca/persistencia/`), los **reportes** (`biblioteca/reportes.py`) y la **interfaz web** con Flask (`web/`). Los dos patrones viven en el dominio y no dependen de Flask ni de SQLite: por eso se prueban con tests unitarios sin base de datos. Los casos de uso los combinan, y la interfaz web nunca crea clases concretas del dominio por su cuenta.

2. State — Ciclo de vida del Remito

Es el patrón central del TP: la consigna indica que «el ciclo de vida del remito y sus estados será evaluado críticamente».

Problema

Un remito de un préstamo interbibliotecario pasa por **Pendiente → Empaquetado → Enviado → Entregado**. En cada etapa hay una sola operación válida, y cada transición tiene efectos propios: registrar su fecha, sacar los ejemplares de la sede (en tránsito no tienen ubicación), validar que ambas sedes sigan activas antes de despachar, ubicarlos en la sede destino al recibirlos y dejar la trazabilidad en el historial.

Sin el patrón, `Remito` tendría un atributo de texto `estado` y cada uno de sus tres métodos un *if/elif* sobre los cuatro estados: doce combinaciones repartidas en tres métodos. Agregar o cambiar un estado obligaría a revisarlos todos, y es fácil olvidar un caso y permitir, por ejemplo, recibir algo que nunca se despachó.

Solución

Rol del patrón	Clase	Responsabilidad
Contexto	Remito (remito.py)	Guarda el estado actual en _estado y delega empaquetar(), despachar() y recibir(). Expone cambiar_estado(), que registra el historial.
Estado (interfaz)	EstadoRemito (estados_remito.py)	Clase abstracta que declara las tres operaciones; _rechazar() centraliza el error.
Estados concretos	Pendiente, Empaquetado, Enviado, Entregado	Cada uno implementa la única operación que admite y rechaza las demás. Entregado rechaza todo: es el estado final.
Trazabilidad	HistorialRemito (remito.py)	Una entrada por estado: fecha y hora, nombre del estado, sede (None en tránsito) y observación.

Cualquier otra operación en cualquier estado → ReglaNegocioError (no cambia el estado ni deja rastro en el historial)

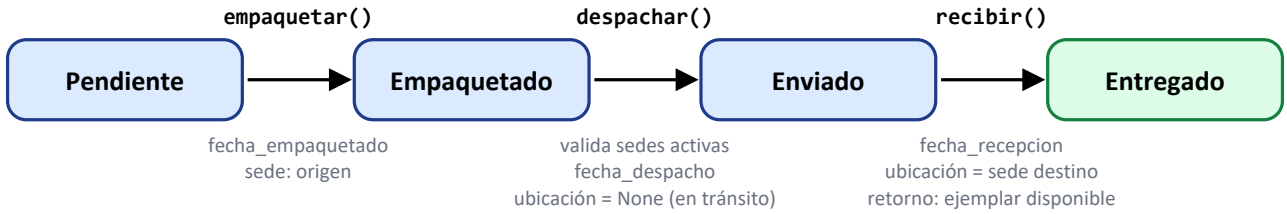


Figura 1. Transiciones del remito y sus efectos. Al crearse, el remito registra en el historial el estado inicial Pendiente.

Implementación

El contexto no decide nada: solo delega en el objeto estado. Todas las transiciones pasan por `cambiar_estado()`, que agrega la entrada de historial.

biblioteca/modelos/remito.py · líneas 55–69

```
55      def empaquetar(self, observacion=""):
56          self._estado.empaquetar(self, observacion)
57
58      def despachar(self, observacion=""):
59          self._estado.despachar(self, observacion)
60
61      def recibir(self, observacion=""):
62          self._estado.recibir(self, observacion)
63
64      def cambiar_estado(self, nuevo_estado, sede_actual, observacion=""):
65          self._estado = nuevo_estado
66          self.registrar_trazabilidad(nuevo_estado.nombre, sede_actual, observacion)
67
68      def registrar_trazabilidad(self, estado, sede, observacion=""):
69          self.historial.append(HistorialRemito(estado, sede, observacion))
```

La interfaz del estado y el estado inicial. Pendiente solo acepta empaquetar(); las otras dos operaciones se rechazan con un mensaje para el usuario:

biblioteca/modelos/estados_remito.py · líneas 14–44

```python
class EstadoRemito(ABC):
    nombre = ""

    @abstractmethod
    def empaquetar(self, remito, observacion=""):
        pass

    @abstractmethod
    def despachar(self, remito, observacion=""):
        pass

    @abstractmethod
    def recibir(self, remito, observacion=""):
        pass

    def _rechazar(self, operacion):
        raise ReglaNegocioError(f"No se puede {operacion} un remito en estado {self.nombre}.")


class Pendiente(EstadoRemito):
    nombre = "Pendiente"

    def empaquetar(self, remito, observacion=""):
        remito.fecha_empaquetado = datetime.now()
        remito.cambiar_estado(Empaquetado(), remito.sede_origen, observacion)

    def despachar(self, remito, observacion=""):
        self._rechazar("despachar")

    def recibir(self, remito, observacion=""):
        self._rechazar("recibir")
```

Despachar y recibir concentran las reglas de negocio de cada etapa: al despachar se verifica que ninguna sede esté dada de baja y los ejemplares quedan en tránsito; al recibir se ubican en la sede destino y, si es el remito de retorno, vuelven a estar disponibles para prestar:

biblioteca/modelos/estados_remito.py · líneas 53–80

```python
    def despachar(self, remito, observacion=""):
        remito.prestamo.validar_sedes_activas()
        remito.fecha_despacho = datetime.now()
        for ejemplar in remito.prestamo.ejemplares():
            ejemplar.ubicacion_actual = None  # en tránsito
        remito.cambiar_estado(Enviado(), None, observacion)

    def recibir(self, remito, observacion=""):
        self._rechazar("recibir")


class Enviado(EstadoRemito):
    nombre = "Enviado"

    def empaquetar(self, remito, observacion=""):
        self._rechazar("empaquetar")

    def despachar(self, remito, observacion=""):
        self._rechazar("despachar")

    def recibir(self, remito, observacion=""):
        destino = remito.sede_destino
        remito.fecha_recepcion = datetime.now()
        for ejemplar in remito.prestamo.ejemplares():
            ejemplar.ubicacion_actual = destino
            if remito.es_retorno():
                ejemplar.marcar_disponible()  # volvió a su sede: se puede prestar otra vez
        remito.cambiar_estado(Entregado(), destino, observacion)
```

Remito de retorno

Cuando el socio devuelve todos los ejemplares en la sede destino, estos vuelven a su sede de pertenencia con un segundo remito (`tipo = Retorno`). **No hizo falta ningún estado nuevo**: el retorno recorre los mismos cuatro estados. Solo se agregó el tipo, y las propiedades `sede_origen` y `sede_destino` invierten el trayecto. Es una muestra concreta de lo que aporta el patrón: el ciclo se reutiliza sin tocar las reglas de transición.

El préstamo también consulta el estado del remito: la devolución solo se admite cuando el remito de ida ya está Entregado, y el retorno solo se genera cuando no quedan ejemplares pendientes:

`biblioteca/modelos/prestamo.py · líneas 149–162`

```
149     def validar_devolucion(self):
150         if self.remito is None or not isinstance(self.remito.estado, Entregado):
151             raise ReglaNegocioError(
152                 f"El material del préstamo {self.id_prestamo} todavía no llegó a la sede destino.")
153 
154     def generar_remito_retorno(self, numero_remito):
155         """Remito de vuelta (destino -> origen), una vez devueltos todos los ejemplares."""
156         if self.remito_retorno is not None:
157             raise ReglaNegocioError(f"El préstamo {self.id_prestamo} ya tiene un remito de retorno.")
158         if self.esta_activo():
159             raise ReglaNegocioError(f"El préstamo {self.id_prestamo} tiene ejemplares sin devolver.")
160         self.validar_sedes_activas()
161         self.remito_retorno = Remito(numero_remito, self, Remito.RETORNO)
162         return self.remito_retorno
```

Justificación

- **Cada regla está en un solo lugar.** Para saber qué pasa si se intenta despachar un remito pendiente basta con leer `Pendiente.despachar`.
- **No se pueden saltear etapas.** El remito no tiene un setter público del estado; la única forma de cambiarlo es que el estado actual llame a `cambiar_estado()`.
- **La trazabilidad no se puede olvidar (RN04).** Como toda transición pasa por `cambiar_estado()`, cada cambio deja su HistorialRemito con fecha, sede y observación.
- **Un intento inválido no deja rastro.** Los rechazos se lanzan antes de modificar nada: el historial y las fechas quedan intactos (lo verifica un test).
- **Alternativas descartadas.** Un *if/elif* sobre un texto (problema descripto arriba) o una tabla de transiciones permitidas: la tabla dice qué transición es válida, pero no resuelve que cada una tenga efectos distintos (fechas, ubicación, validación de sedes).

Consecuencias

- **Ventajas:** se puede agregar un estado (por ejemplo «Extraviado», del opcional de revisión de recepción) creando una clase, sin modificar las existentes; el ciclo de vida coincide uno a uno con el diagrama de clases.
- **Desventajas:** hay más clases, y cada estado repite los métodos que rechaza (se mitigó con `_rechazar()` en la clase base). Cada estado conoce a su sucesor; es aceptable porque el flujo es lineal y fijo.
- **Persistencia:** en la base se guarda el nombre del estado y el Repositorio reconstruye el objeto estado correspondiente (ver sección 6).

Verificación

`test_ciclo_completo_con_trazabilidad_y_ubicacion`, `test_transiciones_no_permitidas`, `test_no_se_despacha_hacia_sede_dada_de_baja`, la clase `TestRemitoRetorno` (`tests/test_modelos.py`) y `test_transicion_invalida_desde_la_base` (`tests/test_persistencia.py`).

3. State — Ciclo de vida de la Reserva

Problema

Una reserva nace **Pendiente**, se **confirma** y termina **cumplida** (el socio retiró el libro) o **cancelada**, ya sea desde Pendiente o desde Confirmada. Confirmar exige que el socio esté habilitado y la sede activa (RN14). A diferencia del remito, el ciclo **se bifurca** y tiene dos estados finales.

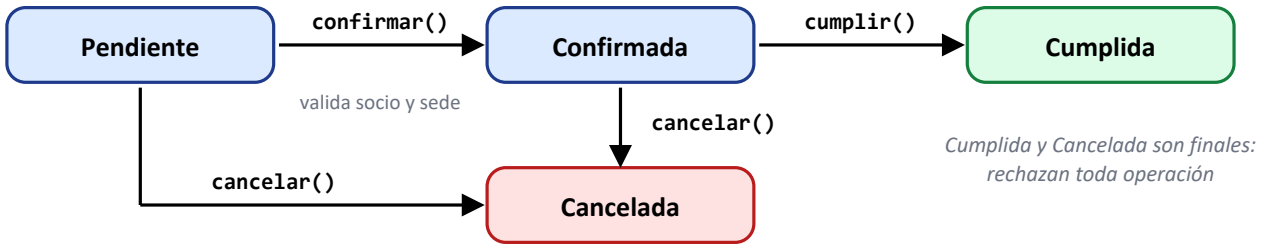


Figura 2. Estados de la reserva.

Solución

Rol del patrón	Clase	Responsabilidad
Contexto	Reserva (reserva.py)	Delega confirmar_reserva(), cancelar() y cumplir() en su estado.
Estado (interfaz)	EstadoReserva (estados_reserva.py)	Declara las tres operaciones.
Estados concretos	ReservaPendiente, ReservaConfirmada, ReservaCumplida, ReservaCancelada	Implementan las transiciones válidas; los dos estados finales rechazan todo.

Implementación

biblioteca/modelos/estados_reserva.py · líneas 32–56

```
32  class ReservaPendiente(EstadoReserva):
33      nombre = "Pendiente"
34
35      def confirmar(self, reserva):
36          reserva.validar_reserva(reserva.socio, reserva.sede)
37          reserva.cambiar_estado(ReservaConfirmada())
38
39      def cancelar(self, reserva):
40          reserva.cambiar_estado(ReservaCancelada())
41
42      def cumplir(self, reserva):
43          self._rechazar("cumplir")
44
45
46  class ReservaConfirmada(EstadoReserva):
47      nombre = "Confirmada"
48
49      def confirmar(self, reserva):
50          self._rechazar("confirmar")
51
52      def cancelar(self, reserva):
53          reserva.cambiar_estado(ReservaCancelada())
54
55      def cumplir(self, reserva):
56          reserva.cambiar_estado(ReservaCumplida())
```

biblioteca/modelos/reserva.py · líneas 32–39

```python
    def confirmar_reserva(self):
        self._estado.confirmar(self)

    def cancelar(self):
        self._estado.cancelar(self)

    def cumplir(self):
        """El socio retiró el libro reservado."""
```

Justificación

- **La validación está solo donde corresponde.** Únicamente `ReservaPendiente.confirmar` llama a `validar_reserva`: cancelar una reserva de un socio que luego fue inhabilitado sigue siendo posible, lo cual es correcto.
- **Un mismo mecanismo para todo el equipo.** Remito y Reserva resuelven su ciclo de vida igual, y la interfaz web solo necesita saber qué botones mostrar según el nombre del estado.
- **El patrón también sirve para ciclos no lineales:** la cancelación desde dos estados y los dos estados finales no requieren ninguna condición extra.

Consecuencias y verificación

Las mismas que en la sección 2: más clases a cambio de reglas localizadas y fáciles de extender. Tests: `test_ciclo_confirmar_y_cumplir`, `test_no_se_cumple_sin_confirmar`, `test_cancelar`, `test_confirmar_reserva`, `test_reserva_rechazada` y `test_reserva` (con la base de datos).

4. Iterator — Recorrido del Catálogo

Problema

El catálogo reúne los ejemplares de todas las sedes de la red. Hay que recorrerlo completo, y también solo los ejemplares **disponibles que pertenecen a una sede**: es la consulta de disponibilidad (proceso obligatorio 6) y la lista de la que se eligen los ejemplares al registrar un préstamo.

Si `Catalogo` expusiera su lista interna, cada cliente repetiría el filtro «disponible y de esta sede», y además podría agregar ejemplares directamente a la lista, salteando el control de códigos duplicados de `agregar_ejemplar()` (RN07).

Solución

Rol del patrón	Clase	Responsabilidad
Iterador (interfaz)	Iterador	Declara has_next() y next(), como en el diagrama de clases.
Iterador concreto	IteradorEjemplares	Recorre todos los ejemplares y guarda su propia posición.
Iterador concreto	IteradorEjemplaresDisponibles	Hereda del anterior y solo redefine has_next() para saltear los que no están disponibles o son de otra sede.
Agregado	Catalogo	Guarda la lista privada _ejemplares y crea los iteradores.
Cliente	servicios.consultar_disponibilidad	Usa el iterador sin conocer la lista.

Implementación

biblioteca/modelos/catalogo.py · líneas 8–54

```python
class Iterador(ABC):
    @abstractmethod
    def has_next(self):
        pass

    @abstractmethod
    def next(self):
        pass

    # Protocolo de Python, para poder usar el iterador en un for.
    def __iter__(self):
        return self

    def __next__(self):
        return self.next()


class IteradorEjemplares(Iterador):
    def __init__(self, ejemplares):
        self._ejemplares = ejemplares
        self._posicion = 0

    def has_next(self):
        return self._posicion < len(self._ejemplares)

    def next(self):
        if not self.has_next():
            raise StopIteration
        ejemplar = self._ejemplares[self._posicion]
        self._posicion += 1
        return ejemplar


class IteradorEjemplaresDisponibles(IteradorEjemplares):
    """Recorre solo los ejemplares disponibles que pertenecen a una sede."""

    def __init__(self, ejemplares, sede):
        super().__init__(ejemplares)
        self._sede = sede

    def has_next(self):
        while self._posicion < len(self._ejemplares):
            ejemplar = self._ejemplares[self._posicion]
            if ejemplar.disponible and ejemplar.pertenece_a_sede(self._sede):
                return True
            self._posicion += 1
        return False
```

biblioteca/modelos/catalogo.py · líneas 57–73

```python
class Catalogo:
    def __init__(self):
        self._ejemplares = []

    def agregar_ejemplar(self, ejemplar):
        if any(e.codigo == ejemplar.codigo for e in self._ejemplares):
            raise ReglaNegocioError(f"Ya existe un ejemplar con código {ejemplar.codigo}.")
        self._ejemplares.append(ejemplar)

    def crear_iterador(self):
        return IteradorEjemplares(self._ejemplares)

    def crear_iterador_disponibles(self, sede):
        return IteradorEjemplaresDisponibles(self._ejemplares, sede)

    def __iter__(self):
        return self.crear_iterador()
```

El cliente, la consulta de disponibilidad que usan la web y el formulario de préstamos:

biblioteca/servicios.py · líneas 85–88

```python
def consultar_disponibilidad(repo, id_sede, isbn=None):
    """Ejemplares disponibles de una sede, recorridos con el iterador del Catalogo."""
    catalogo = repo.catalogo(isbn or None)
    return list(catalogo.crear_iterador_disponibles(repo.sede(id_sede)))
```

Justificación

- **Encapsulamiento:** la colección es privada y la única forma de agregar es `agregar_ejemplar()`, que impide códigos repetidos.
- **El filtro está en un solo lugar.** «Disponible y perteneciente a la sede» se define una vez, en `IteradorEjemplaresDisponibles.has_next()`.
- **Recorridos independientes:** cada iterador tiene su propia posición, así que se pueden hacer varios recorridos simultáneos sin interferir.
- **Interfaz de la cátedra y de Python a la vez:** se respeta `has_next()` / `next()` con `StopIteration` al agotarse, y la interfaz además implementa `__iter__` y `__next__`, de modo que el iterador funciona en un `for` o con `list()`.

Consecuencias

- **Ventajas:** se pueden agregar recorridos nuevos (por ejemplo, por libro o por estado físico) con otra subclase, sin tocar Catalogo ni sus clientes.
- **Desventajas:** en Python las listas ya son iterables, así que el patrón agrega clases; se justifica por el filtrado y por ocultar la colección. El filtro se hace en memoria: con un catálogo muy grande convendría filtrar directamente en SQL.

Verificación

`test_recorre_todos_los_ejemplares_y_lanza_stop_iteration`, `test_se_puede_usar_en_for`, `test_rechaza_ejemplar_duplicado`, `test_solo_disponibles_de_la_sede` y `test_prestamo_local_y_disponibilidad`.

5. Cómo colaboran los patrones en un caso de uso

Los patrones no están aislados. Este es el recorrido de un préstamo interbibliotecario completo, tal como se ejecuta desde la interfaz web:

Paso	Qué ocurre	Patrones
1. Consultar disponibilidad	El formulario lista los ejemplares disponibles de la sede de origen.	Iterator
2. Registrar préstamo	Como las sedes son distintas, se crea un PrestamoInterbibliotecario con vencimiento a 21 días; el constructor valida las reglas; se genera el remito en estado Pendiente con su primera entrada de historial.	State
3. Empaquetar, despachar, recibir	El estado actual decide si la operación es válida y ejecuta sus efectos; cada transición queda registrada en el historial del remito.	State
4. Devolución	Solo se admite si el remito de ida está Entregado; el ejemplar sigue no disponible. Al devolver todo se genera el remito de retorno.	State
5. Retorno	El remito de retorno recorre los mismos estados; al recibirse, los ejemplares quedan disponibles en su sede y vuelven a aparecer en el paso 1.	State, Iterator

6. Técnicas complementarias

No son patrones exigidos por la cátedra, pero sostienen a los anteriores al guardar los datos en SQLite:

- **Repositorio** (`biblioteca/persistencia/repositorio.py`): concentra todo el SQL. Ni el dominio ni la web escriben consultas para leer o guardar objetos.
- **Mapa de identidad:** dentro de una petición, cada fila se convierte en un único objeto. Es necesario porque el dominio compara sedes por identidad (`is`), por ejemplo en `Ejemplar.pertenece_a_sede()`.
- **Reconstrucción del estado (State persistido):** en la base se guarda el nombre del estado; al leer, un diccionario lo traduce a la clase de estado correspondiente:

biblioteca/persistencia/repositorio.py · líneas 18–20

```
18  ESTADOS_REMITO = {c.nombre: c for c in (Pendiente, Empaquetado, Enviado, Entregado)}
19  ESTADOS_RESERVA = {c.nombre: c for c in (ReservaPendiente, ReservaConfirmada,
20                                           ReservaCumplida, ReservaCancelada)}
```

biblioteca/persistencia/repositorio.py · líneas 260–264

```
260          r = Remito.__new__(Remito)
261          r.numero_remito = f["numero_remito"]
262          r.prestamo = prestamo
263          r.tipo = f["tipo"]
264          r._estado = ESTADOS_REMITO[f["estado"]]()
```

Además, las restricciones de la base (claves primarias, UNIQUE y un índice único parcial que impide que un ejemplar tenga dos préstamos pendientes) funcionan como última barrera para las reglas de no duplicación, por si alguna validación se saltea por error.